

Submission: hw01

Zidane Karim <zkarim7676@gmail.com>

11 Source Files

2 Plots

597 Lines

Generated: 2026-09-09 17:01:34

Source Code & Configuration

pyproject.toml

31 lines

```
1  # pyproject.toml.jinja
2
3  [project]
4  name = "hw01"
5  version = "0.1.0"
6  description = "Linear Regression of Noisy Sin Wave"
7  readme = "README.md"
8  authors = [
9      { name = "Zidane Karim", email = "zidane.karim@cooper.edu" }
10 ]
11 requires-python = ">=3.13"
12 dependencies = [
13     "structlog",
14     "rich",
15     "numpy",
16     "pydantic",
17     "pydantic-settings>=2.10.1",
18     "matplotlib>=3.10.5",
19     "tqdm>=4.67.1",
20     "jax[cpu,cuda12]>=0.7.1",
21     "flax>=0.11.2",
22     "optax>=0.2.4",
23     "scikit-learn",
24 ]
25
26 [project.scripts]
27 hw01 = "hw01:main"
28 hw01-evaluate = "hw01.evaluate:main"
29 [build-system]
30 requires = ["uv_build>=0.8.3,<0.9.0"]
31 build-backend = "uv_build"
```

README.md

Markdown

4 lines

hw01

Linear Regression of Noisy Sin Wave

Minimizes loss using Gaussian basis functions

src/hw01/config.toml

20 lines

```
1  debug = false
2  random_seed = 0xc61dd8fc
3
4  [data]
5  num_features = 1
6  num_samples = 50
7  sigma_noise = 0.1 # changed from 0.5
8
9  [training]
10 batch_size = 16
11 num_iters = 5000
12 learning_rate = 0.1
13 M = 6
14
15
16
17 [plotting]
18 output_dir = "artifacts"
19 figsize = [5, 3]
20 dpi = 200
```

src/hw01/__init__.py

64 lines

```

1  import jax
2  import numpy as np
3  import optax
4  import structlog
5  from flax import nnx
6
7  from .config import load_settings
8  from .data import Data
9  from .logging import configure_logging
10 from .model import LinearModel, NNXLinearModel, NNXRegressionModel
11 from .plotting import compare_linear_models, plot_fit, plot_basis_functions
12 from .training import train
13
14
15 def main() -> None:
16     """CLI entry point."""
17     settings = load_settings()
18     configure_logging()
19     log = structlog.get_logger()
20     log.info("Settings loaded", settings=settings.model_dump())
21
22     # JAX PRNG
23     key = jax.random.key(settings.random_seed)
24     data_key, model_key = jax.random.split(key)
25     np_rng = np.random.default_rng(np.asarray(jax.random.key_data(data_key)))
26
27     data_generating_model = LinearModel(
28         weights=np_rng.integers(low=0, high=5, size=(settings.data.num_features)),
29         bias=2,
30     )
31     # I couldn't figure out how to adjust these weights and biases--inside the data.py, these
are not really used since the clean_y is generated by np.sin
32     log.debug("Data generating model", model=data_generating_model)
33
34     data = Data(
35         model=data_generating_model,
36         rng=np_rng,
37         num_features=settings.data.num_features,
38         num_samples=settings.data.num_samples,
39         sigma=settings.data.sigma_noise,
40     )
41
42     model = NNXRegressionModel(
43         rngs=nnx.Rngs(params=model_key),
44         num_features=settings.data.num_features,
45         M=settings.training.M,
46     )
47     log.debug("Initial model", model=model.model)
48
49     optimizer = nnx.Optimizer(
50         model, optax.adam(settings.training.learning_rate), wrt=nnx.Param
51     )
52
53     train(model, optimizer, data, settings.training, np_rng)
54
55     log.debug("Trained model", model=model.model)

```

```
56
57     # compare_linear_models(data.model, model.model)
58     # I commented out this line since it wasn't properly reflecting my synthetic data, and it
wasn't in the assignment description
59
60     if settings.data.num_features == 1:
61         plot_fit(model, data, settings.plotting)
62         plot_basis_functions(model.basis, settings.plotting)
63     else:
64         log.info("Skipping plotting for multi-feature models.")
```

src/hw01/config.py

99 lines

```
1  from pathlib import Path
2  from importlib.resources import files
3  from typing import Tuple
4
5  from pydantic import BaseModel, Field
6  from pydantic_settings import (
7      BaseSettings,
8      PydanticBaseSettingsSource,
9      SettingsConfigDict,
10     TomlConfigSettingsSource,
11 )
12
13
14 def _find_config_toml() -> Path:
15     """Resolve config.toml location via importlib.resources or relative path fallback."""
16     if __package__:
17         try:
18             return Path(str(files(__package__).joinpath("config.toml")))
19         except Exception:
20             pass
21     return Path(__file__).parent / "config.toml"
22
23
24 class DataSettings(BaseModel):
25     """Settings for data generation."""
26
27     num_features: int = Field(
28         default=1, gt=0, description="Number of feature dimensions"
29     )
30     num_samples: int = Field(
31         default=50, gt=0, description="Number of synthetic data points"
32     )
33     sigma_noise: float = Field(
34         default=0.5, ge=0.0, description="Standard deviation of Gaussian noise"
35     )
36
37
38 class TrainingSettings(BaseModel):
39     """Settings for model training."""
40
41     batch_size: int = Field(default=16, gt=0, description="Mini-batch size for SGD")
42     num_iters: int = Field(
43         default=300, gt=0, description="Number of optimization iterations"
44     )
45     learning_rate: float = Field(
46         default=0.1, gt=0.0, description="Learning rate for Adam optimizer"
47     )
48     M: int = Field(default=6, gt=0, description="number of gaussian curves")
49
50
51 class PlottingSettings(BaseModel):
52     """Settings for plotting."""
53
54     figsize: Tuple[int, int] = (5, 3)
55     dpi: int = Field(default=200, gt=0, description="Dots per inch for saved figures")
56     output_dir: Path = Path("artifacts")
```

```
57
58
59 class AppSettings(BaseSettings):
60     """Main application settings."""
61
62     debug: bool = False
63     random_seed: int = 31415
64     data: DataSettings = DataSettings()
65     training: TrainingSettings = TrainingSettings()
66     plotting: PlottingSettings = PlottingSettings()
67
68     model_config = SettingsConfigDict(
69         toml_file=_find_config_toml(),
70         env_nested_delimiter="__",
71         extra="ignore",
72     )
73
74     @classmethod
75     def settings_customise_sources(
76         cls,
77         settings_cls: type[BaseSettings],
78         init_settings: PydanticBaseSettingsSource,
79         env_settings: PydanticBaseSettingsSource,
80         dotenv_settings: PydanticBaseSettingsSource,
81         file_secret_settings: PydanticBaseSettingsSource,
82     ) -> tuple[PydanticBaseSettingsSource, ...]:
83         """
84         Set the priority of settings sources.
85
86         We use a TOML file for configuration.
87         """
88         return (
89             init_settings,
90             TomlConfigSettingsSource(settings_cls),
91             env_settings,
92             dotenv_settings,
93             file_secret_settings,
94         )
95
96
97 def load_settings() -> AppSettings:
98     """Load application settings."""
99     return AppSettings()
```

src/hw01/data.py

41 lines

```

1  from dataclasses import InitVar, dataclass, field
2
3  import numpy as np
4
5  from .model import LinearModel
6
7
8  @dataclass
9  class Data:
10     """Handles generation of synthetic data for linear regression."""
11
12     model: LinearModel
13     rng: InitVar[np.random.Generator]
14     num_features: int
15     num_samples: int
16     sigma: float
17     x: np.ndarray = field(init=False)
18     y: np.ndarray = field(init=False)
19     index: np.ndarray = field(init=False)
20
21     def __post_init__(self, rng: np.random.Generator):
22         """Generate synthetic data based on the model."""
23         self.index = np.arange(self.num_samples)
24         self.x = rng.uniform(
25             0, 1.0, size=(self.num_samples, self.num_features)
26         ) # manually changed from (0.1, 0.9) to (0, 1), to reflect assignment specs
27         # clean_y = self.x @ self.model.weights[:, np.newaxis] + self.model.bias
28         # I changed clean_y from the weights + bias here, to the sin wave output.
29         # This is because in training:loss_fn, the y^ is already taken from the model, so I
30         # didn't really understand why I should basically replicate that in this step. emailed Prof Curro about this
31         clean_y = np.sin(2 * np.pi * self.x)
32         self.y = rng.normal(
33             loc=clean_y, scale=self.sigma
34         ).flatten() # this produces the + epsilon_i from my understanding
35
36     def get_batch(
37         self, rng: np.random.Generator, batch_size: int
38     ) -> tuple[np.ndarray, np.ndarray]:
39         """Select random subset of examples for training batch."""
40         choices = rng.choice(self.index, size=batch_size)
41
42         return self.x[choices], self.y[choices].flatten()

```

src/hw01/model.py

95 lines

```

1  from dataclasses import dataclass
2
3  import jax
4  import jax.numpy as jnp
5  import numpy as np
6  from flax import nnx
7  # Credit to Josh Miao, EE '27 for helping me understand the problem, and Ahikara Sandrasagra EE
'28, for providing me with the material to understand
8
9
10 @dataclass
11 class LinearModel:
12     """Represents a simple linear model."""
13
14     weights: np.ndarray
15     bias: float
16
17
18 @dataclass
19 class Parameters: # avoided calling this simple BasisExpansion since it's not really the same
20     mu: np.ndarray # n-dimensional array
21     sigma: np.ndarray
22     weights: np.ndarray
23     bias: float
24
25
26 class NNLinearModel(nnx.Module):
27     """A Flax NN module for a linear regression model."""
28
29     def __init__(self, *, rngs: nnx.Rngs, num_features: int):
30         self.num_features = num_features
31         key = rngs.params()
32         self.w = nnx.Param(jax.random.normal(key, (self.num_features, 1)))
33         self.b = nnx.Param(jnp.zeros((1, 1)))
34
35     def __call__(self, x: jax.Array) -> jax.Array:
36         """Predicts the output for a given input."""
37         return jnp.squeeze(x @ self.w.value + self.b.value, axis=-1)
38
39     @property
40     def model(self) -> LinearModel:
41         """Returns the underlying simple linear model."""
42         return LinearModel(
43             weights=np.array(self.w.value).reshape([self.num_features]),
44             bias=float(np.array(self.b.value).squeeze()),
45         )
46
47
48 class NNXBasisExpansion(nnx.Module):
49     def __init__(self, *, rngs: nnx.Rngs, num_features: int, M: int):
50         self.num_features = num_features
51         self.M = M
52         key = rngs.params()
53         # src: https://flax.readthedocs.io/en/stable/api_reference/flax.nnx/variables.html
54         # param required for learnable & automatic diff
55         self.mu = nnx.Param(

```

```

56         jnp.linspace(0.0, 1.0, num=self.M)
57     ) # had to ask about min/max, but makes sense since we are sampling x from (0,1)
58     # log sig here cuz theyll die out otherwise, thanks Josh Miao
59     self.sig = nnx.Param(
60         jnp.ones((self.M,)) * jnp.log((1.0 / self.M))
61     ) # 0.16? let M=6, 1/M
62     # sig uses ones so no key needed
63
64     def __call__(self, x: jax.Array):
65         """Predicts the output for a given input.""" # copied from NNLinearModel
66         # to ensure broadcasting we need to reshape x here
67         if x.ndim == 1:
68             x = x[
69                 :, jnp.newaxis
70             ] # adds singleton axis (equivalent to None, but dummy axis essentially)
71         # restore sig when we are computing phi
72         return jnp.exp(-((x - self.mu.value) ** 2) / jnp.exp(self.sig.value) ** 2)
73
74
75     class NNRegressionModel(nnx.Module):
76         def __init__(self, *, rngs: nnx.Rngs, num_features: int, M: int):
77             self.M = M
78             self.basis = NNxBasisExpansion(rngs=rngs, num_features=num_features, M=self.M)
79             self.linear = NNLinearModel(rngs=rngs, num_features=self.M)
80
81         def __call__(self, x: jax.Array) -> jax.Array:
82             phi_x = self.basis(x)
83             return self.linear(
84                 phi_x
85             ) # we can pass phi_x in place of x as according to given functional form
86             # and we need to read from this regression rather than the straight linear
87
88     @property
89     def model(self) -> Parameters:
90         return Parameters(
91             mu=np.array(self.basis.mu.value),
92             sigma=np.array(self.basis.sig.value),
93             weights=np.array(self.linear.w.value).reshape([self.linear.num_features]),
94             bias=float(np.array(self.linear.b.value).squeeze()),
95         )

```

src/hw01/training.py

50 lines

```
1  import jax.numpy as jnp
2  import numpy as np
3  import structlog
4  from flax import nnx
5  from tqdm import trange
6
7  from .config import TrainingSettings
8  from .data import Data
9  from .model import NNLinearModel, NNRegressionModel
10
11  log = structlog.get_logger()
12
13
14  @nnx.jit
15  def train_step(
16      model: NNRegressionModel,
17      optimizer: nnx.Optimizer,
18      x: jnp.ndarray,
19      y: jnp.ndarray, # changed here to NNRegressionModel
20  ):
21      """Performs a single training step."""
22
23      def loss_fn(model: NNRegressionModel):
24          y_hat = model(x)
25          return 0.5 * jnp.mean((y_hat - y) ** 2)
26
27      loss, grads = nnx.value_and_grad(loss_fn)(model)
28      optimizer.update(model, grads) # In-place update of model parameters
29      return loss
30
31
32  def train(
33      model: NNRegressionModel,
34      optimizer: nnx.Optimizer,
35      data: Data,
36      settings: TrainingSettings,
37      np_rng: np.random.Generator,
38  ) -> None:
39      """Train the model using SGD."""
40      log.info("Starting training", **settings.model_dump())
41      bar = trange(settings.num_iters)
42      for i in bar:
43          x_np, y_np = data.get_batch(np_rng, settings.batch_size)
44          x, y = jnp.asarray(x_np), jnp.asarray(y_np)
45
46          loss = train_step(model, optimizer, x, y)
47
48          bar.set_description(f"Loss @ {i} => {loss:.6f}")
49          bar.refresh()
50      log.info("Training finished")
```

src/hw01/evaluate.py

14 lines

```
1  """Evaluation entry point for hw01."""
2
3  import structlog
4
5  from .logging import configure_logging
6
7
8  def main() -> None:
9      """CLI entry point for homework evaluation."""
10     configure_logging()
11     log = structlog.get_logger()
12     log.info(
13         "Evaluation not yet implemented for hw01. Implement evaluation logic in src/hw01/
evaluate.py"
14     )
```

src/hw01/plotting.py

116 lines

```

1  import matplotlib
2  import matplotlib.pyplot as plt
3  import matplotlib.style
4  import jax.numpy as jnp
5  import numpy as np
6  import structlog
7
8  from .config import PlottingSettings
9  from .data import Data
10 from .model import LinearModel, NNLinearModel, NNRegressionModel, NNxBasisExpansion
11
12 log = structlog.get_logger()
13
14 font = {
15     # "family": "Adobe Caslon Pro",
16     "size": 10,
17 }
18
19 matplotlib.style.use("classic")
20 matplotlib.rc("font", **font)
21 plt.rcParams["svg.fonttype"] = "none"
22
23
24 def compare_linear_models(a: LinearModel, b: LinearModel):
25     """Prints a comparison of two linear models."""
26     log.info("Comparing models", true=a, estimated=b)
27     print("w,    w_hat")
28     for w_a, w_b in zip(a.weights, b.weights):
29         print(f"{w_a:0.2f}, {w_b:0.2f}")
30
31     print(f"{a.bias:0.2f}, {b.bias:0.2f}")
32
33
34 def plot_fit(
35     model: NNRegressionModel, # changed to Regression model here
36     data: Data,
37     settings: PlottingSettings,
38 ):
39     """Plots the linear fit and saves it to a file."""
40     log.info("Plotting fit")
41     fig, ax = plt.subplots(1, 1, figsize=settings.figsize, dpi=settings.dpi)
42
43     ax.set_title("Regression Fit")
44     ax.set_xlabel("x")
45     ax.set_ylim(np.amax(data.y) * -1.5, np.amax(data.y) * 1.5) # added negative y-lim
46     h = ax.set_ylabel("y", labelpad=10)
47     h.set_rotation(0)
48
49     xs = np.linspace(0, 1, 100)
50     xs = xs[:, np.newaxis]
51
52     true_sin_y = np.sin(2 * np.pi * xs)
53     # true sin plot
54     ax.plot(
55         xs, true_sin_y, color="red", linestyle="--", linewidth=1.5, label="True Sin"
56     )

```

```

57
58     # noisy sin plot (same as true, but with sigma noise added)
59     ax.plot(
60         np.squeeze(data.x),
61         data.y,
62         "o",
63         color="green",
64         markeredgecolor="black",
65         label="Noisy Sin",
66     )
67
68     # regression plot
69     ax.plot(
70         xs,
71         np.squeeze(model(jnp.asarray(xs[:, np.newaxis]))),
72         "-",
73         color="blue",
74         label="Regression",
75     )
76
77     plt.tight_layout()
78     plt.legend()
79     settings.output_dir.mkdir(parents=True, exist_ok=True)
80     svg_path = settings.output_dir / "fit.svg"
81     pdf_path = settings.output_dir / "fit.pdf"
82     plt.savefig(svg_path)
83     plt.savefig(pdf_path)
84     plt.close(fig)
85     log.info("Saved plot", path=str(svg_path))
86
87
88     def plot_basis_functions(
89         model: NNXBasisExpansion, # probably? also i had to ask someone what plotting a basis
expansion meant and I learned it was just phi's
90         settings: PlottingSettings,
91     ):
92         print("Learned mus:\n", np.sort(np.array(model.mu.value)))
93         print("Learned (log) sigmas:\n", np.array(model.sig.value))
94         """Plots M Gaussian Basis Functions and saves it to a file."""
95         log.info("Plotting bases")
96         fig, ax = plt.subplots(1, 1, figsize=settings.figsize, dpi=settings.dpi)
97
98         ax.set_title("Bases for fit")
99         ax.set_xlabel("x")
100         ax.set_ylim(0, 1.0)
101         h = ax.set_ylabel("y", labelpad=10)
102         h.set_rotation(0)
103
104         xs = np.linspace(-2, 2, 1000)
105         xs = xs[:, np.newaxis]
106         ax.plot(xs, np.squeeze(model(jnp.asarray(xs))), "-")
107
108         plt.tight_layout()
109
110         settings.output_dir.mkdir(parents=True, exist_ok=True)
111         svg_path = settings.output_dir / "bases.svg"
112         pdf_path = settings.output_dir / "fit.pdf"
113         plt.savefig(svg_path)
114         # plt.savefig(pdf_path)

```

```
115 plt.close(fig)
116 log.info("Saved plot", path=str(svg_path))
```

src/hw01/logging.py

63 lines

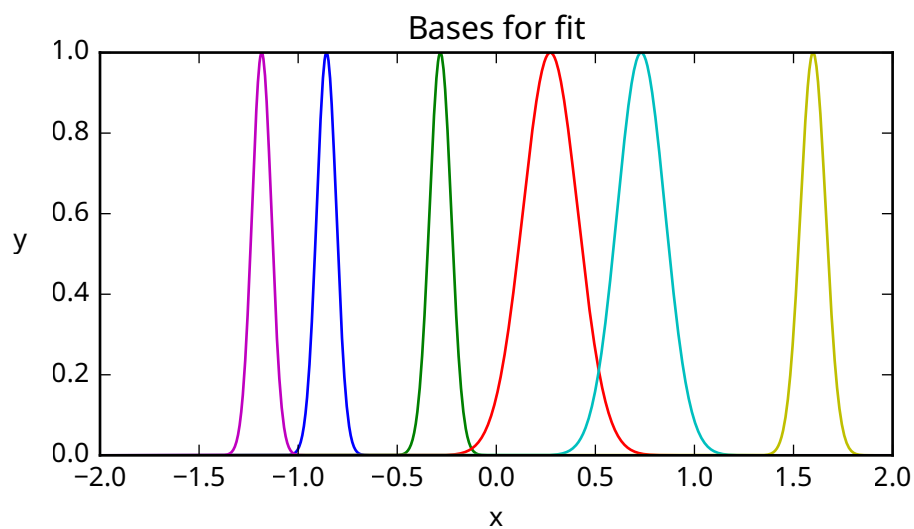
```
1  import logging
2  import os
3  import sys
4  from pathlib import Path
5
6  import jax
7  import numpy as np
8  import structlog
9
10
11  class FormattedFloat(float):
12      def __repr__(self) -> str:
13          return f"{self:.4g}"
14
15
16  def custom_serializer_processor(logger, method_name, event_dict):
17      for key, value in event_dict.items():
18          # Handle JAX arrays in addition to TF tensors
19          if hasattr(value, "numpy"): # Covers TF tensors
20              value = value.numpy()
21          if isinstance(value, jax.Array):
22              value = np.array(value)
23          if isinstance(value, (np.generic, np.ndarray)):
24              value = value.item() if value.size == 1 else value.tolist()
25          if isinstance(value, float):
26              value = FormattedFloat(value)
27          if isinstance(value, Path):
28              value = str(value)
29          event_dict[key] = value
30      return event_dict
31
32
33  def configure_logging():
34      """Configure logging for the application."""
35      log_level = os.environ.get("LOG_LEVEL", "INFO").upper()
36
37      logging.basicConfig(
38          format="%(message)s",
39          stream=sys.stdout,
40          level=log_level,
41          force=True,
42      )
43      logging.getLogger().setLevel(log_level)
44      structlog.configure(
45          processors=[
46              structlog.stdlib.filter_by_level,
47              structlog.stdlib.add_logger_name,
48              structlog.stdlib.add_log_level,
49              structlog.stdlib.PositionalArgumentsFormatter(),
50              structlog.processors.TimeStamper(fmt="iso"),
51              structlog.processors.StackInfoRenderer(),
52              structlog.processors.format_exc_info,
53              structlog.processors.UnicodeDecoder(),
54              custom_serializer_processor,
55              structlog.dev.ConsoleRenderer(
56                  colors=True, exception_formatter=structlog.dev.RichTracebackFormatter()
```

```
57         ),
58     ],
59     context_class=dict,
60     logger_factory=structlog.stdlib.LoggerFactory(),
61     wrapper_class=structlog.stdlib.BindLogger,
62     cache_logger_on_first_use=True,
63 )
```

Artifacts & Generated Vector Plots

artifacts/bases.svg

Vector SVG



artifacts/fit.svg

Vector SVG

